

Programming for Business (CL-2016)

LAB INSTRUCTOR
OWAIS HAMEED

Week 06 Objectives:

- Why functions
- Function definition and parameters
- Return values
- Parameters and Arguments
- Positional and keyword argument
- Default arguments
- Scope of variables
- Lambda function

Functions:

- It is a self-contained block of code designed to perform a specific task. You can think of it as a "mini-program" inside your main program.
- Functions in business software applications perform specific tasks like data analysis, financial management, and customer relationship management(CRM).
- They streamline processes, automate tasks, improve efficiency, and provide real-time insights, enhancing productivity and decision-making across industries.

The "Input-Process-Output" Model

- Every function follows a predictable three-step flow:
- **Input (Parameters):** The raw data you feed into the machine (e.g., your Account Number and the amount you want to withdraw).
- **Process (Logic):** The internal steps the machine takes (e.g., checking if you have enough balance, subtracting the amount, and logging the time).
- **Output (Return Value):** The result the machine hands back to you (e.g., the new balance or a "Transaction Successful" message).

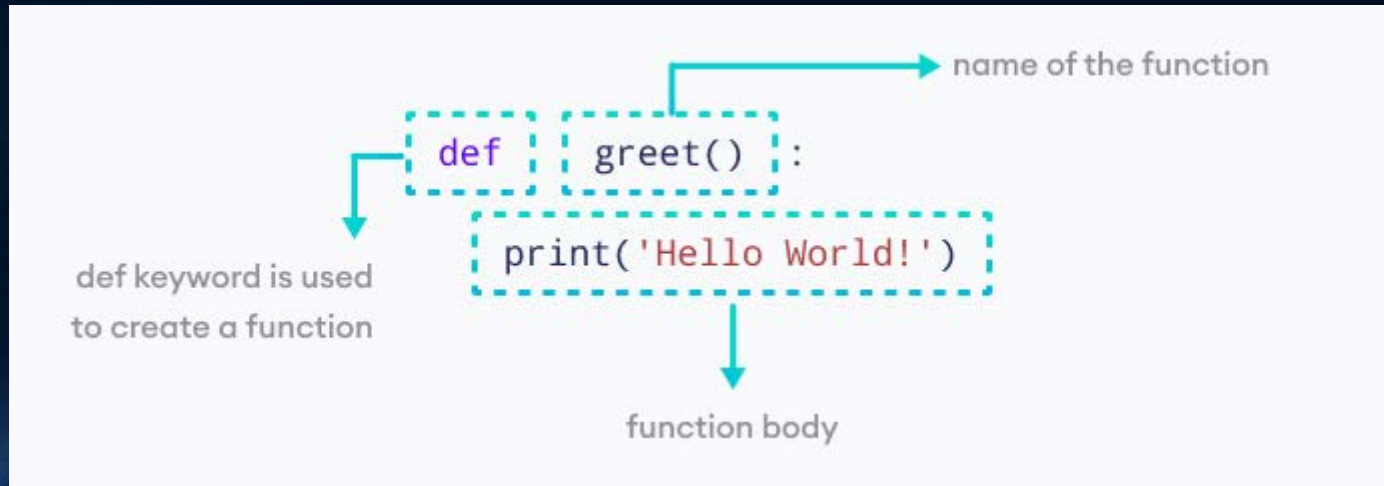
Importance

- The importance of these functions lies in their ability to simplify tasks, minimize errors, code reusability and improve overall efficiency.
- Functions act as "Gatekeepers." By using functions, you control how data is modified.
- By leveraging these functions, businesses can streamline operations, enhance customer satisfaction, optimize resource allocation, and gain a competitive edge in the market.

Defining a Function

- We can define a function in Python, using the **def** keyword. A function might take input in the form of parameters.
- The syntax to declare a function is:

```
def greet():  
    print('Hello World!')
```



Example

Here, we define a function using def that prints a welcome message when called.

- `def depart():`
- `print("Welcome to FSM")`

No Output

Calling a Function

After creating a function in Python we can call it by using the name of the functions followed by parenthesis containing parameters of that particular function.

- `def depart():`
- `print("Welcome to FSM")`
- `depart()`

- You can call the same function multiple times:

Example

- ```
def my_function():
 print("Hello from a function")
```

```
my_function()
my_function()
my_function()
```



# Function Naming

Function names follow the same rules as variable names in Python:

- A function name must start with a letter or underscore
- A function name can only contain letters, numbers, and underscores
- Function names are case-sensitive (myFunction and myfunction are different)

## Example

`calculate_sum()`

`_private_function()`

`myFunction2()`

# Function to Add Two Numbers

function with two arguments

- `def add_numbers(num1, num2):`
- `sum = num1 + num2`
- `print("Sum: ", sum)`

function call with two values

- `add_numbers(5, 4)`

In the above example, we have created a function named `add_numbers()` with arguments: `num1` and `num2`.

The diagram illustrates the relationship between a function definition and its invocation. It features two code snippets: a function definition at the top and a function call at the bottom. The function definition is `def add_numbers(num1, num2):` followed by `# code`. The function call is `add_numbers(5, 4)` followed by `# code`. A solid blue line connects the function call to the definition, with a '1' at the end of the line pointing to the function name. Two dashed blue arrows originate from the arguments '5' and '4' in the function call and point to the parameters 'num1' and 'num2' in the function definition, respectively. A '2' is placed near the start of the solid line connecting the two code blocks.

```
def add_numbers(num1, num2):
 # code

add_numbers(5, 4)
code
```

# Return Values

- Functions can send data back to the code that called them using the return statement.
- When a function reaches a return statement, it stops executing and sends the result back:

Example

A function that returns a value:

- `def find_square(num):`
- `result = num * num`
- `return result`

function call

- `square = find_square(3)`
- `print('Square:', square)`

```
def find_square(num):
 # code
 return result

square = find_square(3)
code
```

The diagram illustrates the execution flow. A teal arrow labeled '1' originates from the `find_square(3)` call in the function call section and points back to the `def find_square(num):` line in the function definition section. A second teal arrow labeled '2' points downwards from the `return result` line in the function definition to the `find_square(3)` call in the function call section, indicating the return of the value.

## Example

Without functions - repetitive code:

```
temp1 = 77
celsius1 = (temp1 - 32) * 5 / 9
print(celsius1)

temp2 = 95
celsius2 = (temp2 - 32) * 5 / 9
print(celsius2)

temp3 = 50
celsius3 = (temp3 - 32) * 5 / 9
print(celsius3)
```

With functions, you write the code once and reuse it:

## Example

With functions - reusable code:

```
def fahrenheit_to_celsius(fahrenheit):
 return (fahrenheit - 32) * 5 / 9

print(fahrenheit_to_celsius(77))
print(fahrenheit_to_celsius(95))
print(fahrenheit_to_celsius(50))
```

# Returning Different Data Types

- Functions can return any data type, including lists, tuples, dictionaries, and more.

## Example

- A function that returns a list:

```
def my_function():
 return ["apple", "banana", "cherry"]
fruits = my_function()
print(fruits[0])
print(fruits[1])
print(fruits[2])
```

```
my_function()
```



# The pass Statement

- Function definitions cannot be empty. If you need to create a function placeholder without any code, use the pass statement:

## Example

- `def my_function():`
- `pass`

# Arguments

- Information can be passed into functions as arguments.
- Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.
- The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

## Example

A function with one argument:

- ```
def my_function(fname):  
    print(fname + " Elis")  
  
my_function("Tom")  
my_function("Harry")  
my_function("Michael")
```

Parameters vs Arguments

- The terms *parameter* and *argument* can be used for the same thing: information that are passed into a function.
- **From a function's perspective:**
- A **parameter** is the variable listed inside the parentheses in the function definition.
- An **argument** is the actual value that is sent to the function when it is called.

Example

- ```
def my_function(name): # name is a parameter
 print("Hello", name)
```

```
my_function("Emil") # "Emil" is an argument
```

# Number of Arguments

- By default, a function must be called with the correct number of arguments.
- If your function expects 2 arguments, you must call it with exactly 2 arguments.

## Example

- This function expects 2 arguments, and gets 2 arguments::
- ```
def my_function(fname, lname):  
    print(fname + " " + lname)  
  
my_function("Matt", "Renshaw")
```

Default Parameter Values

- You can assign default values to parameters. If the function is called without an argument, it uses the default value:

Example

```
def my_function(name = "friend"):
    print("Hello", name)
```

```
my_function("Emily")
```

```
my_function("Sam")
```

```
my_function()
```

```
my_function("Jack")
```

Keyword Arguments

- You can send arguments with the *key = value* syntax.

Example

- ```
def my_function(name, animal):
 print("I have a", animal)
 print("My", animal + "'s name is", name)
```

```
my_function(animal = "dog", name = "Buddy")
```

# Positional Arguments

- When you call a function with arguments without using keywords, they are called positional arguments.
- Positional arguments must be in the correct order:

## Example

- ```
def my_function(animal, name):  
    print("I have a", animal)  
    print("My", animal + "'s name is", name)  
  
my_function("dog", "Buddy")
```


Mixing Positional and Keyword Arguments

- You can mix positional and keyword arguments in a function call.
- However, positional arguments must come before keyword arguments:

Example

- ```
def my_function(animal, name, age):
 print("I have a", age, "year old", animal, "named", name)

my_function("dog", name = "Buddy", age = 5)
```

# Positional-Only Arguments

You can specify that a function can have ONLY positional arguments.  
To specify positional-only arguments, add `,` `/` after the arguments:

## Example

```
def my_function(name, /):
 print("Hello", name)
```

```
my_function("Emil")
```

# Keyword-Only Arguments

To specify that a function can have only keyword arguments, add `*`, *before* the arguments:

## Example

```
def my_function(*, name):
 print("Hello", name)
```

```
my_function(name = "Emil")
```

# Python Function With Arbitrary Arguments

- Sometimes, we do not know in advance the number of arguments that will be passed into a function. To handle this kind of situation, we can use arbitrary arguments in Python.
- Arbitrary arguments allow us to pass a varying number of values during a function call.
- We use an asterisk (\*) before the parameter name to denote this kind of argument. For example,

In the this example, we have created the function `find_sum()` that accepts arbitrary arguments. Notice the lines,

```
find_sum(1, 2, 3)
```

```
find_sum(4, 9)
```

Here, we are able to call the same function with different arguments.

```
program to find sum of multiple numbers

def find_sum(*numbers):
 result = 0

 for num in numbers:
 result = result + num

 print("Sum = ", result)

function call with 3 arguments
find_sum(1, 2, 3)

function call with 2 arguments
find_sum(4, 9)
```

# Python Local Variables

- When we declare variables inside a function, these variables will have a local scope (within the function). We cannot access them outside the function.
- These types of variables are called local variables. For example,

```
def greet():
 # local variable
 message = 'Hello'

 print('Local', message)

greet()

try to access message variable
outside greet() function
print(message)
```

# Python Global Variables

- In Python, a variable declared outside of the function or in global scope is known as a global variable. This means that a global variable can be accessed inside or outside of the function.
- Let's see an example of how a global variable is created in Python.

```
declare global variable
message = 'Hello'

def greet():
 # declare local variable
 print('Local', message)

greet()
print('Global', message)
```



# Lambda Functions

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.

## Example

Add 10 to argument `a`, and return the result:

```
x = lambda a : a + 10
print(x(5))
```

- Syntax
- lambda arguments : expression
- The expression is executed and the result is returned:

# More examples

Lambda functions can take any number of arguments:

## Example

Multiply argument `a` with argument `b` and return the result:

```
x = lambda a, b : a * b
print(x(5, 6))
```

## Example

Summarize argument `a`, `b`, and `c` and return the result:

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

# Final Lab Task:

1. Write a function that takes string input and reverse a string.

Sample String : "1234abcd"

Expected Output : "dcba4321"

2. Function for EMI calculation
3. Profit calculation function

# Final Lab Task

4. Create a function to calculate savings growth over months.

- Requirements:
- Function `calculate_savings(balance, monthly_saving)`
- Ask user: Monthly saving, Number of months.
- Use loop to add savings each month
- Display balance after each month

# Final Lab Task

5. Create a system to calculate commission for multiple sales.

Requirements

Function commission(amount)

Commission rules:  $< 10000 \rightarrow 5\%$ ,  $10000-50000 \rightarrow 8\%$ ,  $50000 \rightarrow 12\%$

Use loop to ask user if the user want to enter another amount or exit the program and Display:

Commission per sale

Total commission